

χ -VLA: A Fully Tensor-Decomposable Vision-Language-Action Model

Building a robot policy that is a single algebraic object, from pixels to actions

July 16, 2026

Abstract

A modern robot policy such as OpenVLA is a stack of ordinary neural-network blocks: attention layers with a softmax, gated multi-layer perceptrons (MLPs) with smooth activation functions, and normalization layers that rescale each input by a quantity computed from that same input. These operations make the model hard to analyze, because the network cannot be rewritten as one closed-form algebraic expression. We ask what happens if every learned block is instead restricted to operations that are polynomial in the activations: linear maps, multiplications, additions, and fixed rescalings. Under that restriction the whole model becomes a single multilinear object, a tensor network, which can be factored and inspected directly from its weights.

Our observation is that the pieces needed to do this already exist in the literature and that OpenVLA’s modular design is ready to receive them. We combine three ingredients: activation-free “bilinear” feed-forward layers and softmax-free “bilinear” attention (from Riggs’ tensor-transformer work), and a normalization scheme that after training collapses into a fixed constant so it can be absorbed into a neighbouring weight matrix (from Dooms et al.’s χ -nets). We assemble these along OpenVLA’s skeleton to obtain χ -VLA, a policy whose entire learned map from pixels and token identities to a continuous action is one tensor network.

We report five results. (1) The exported model is genuinely a tensor network: it folds to a form with no input-dependent division, reproducing the trained network to within 10^{-6} in single precision and 10^{-12} in double precision. (2) It stays competitive after folding: a softmax-free vision encoder reaches 98.8% of a matched softmax baseline on image classification, and a strict foldable language model matches its non-strict teacher. (3) A cross-bilinear fusion layer beats a plain concatenation baseline by 81 accuracy points on a task where the answer provably requires multiplying the two input streams, and removing the multiplication term erases the gain. (4) An end-to-end tensor-pure policy uses language in a measurable way: shuffling the instruction degrades its predictions. (5) The interpretability payoff that an ordinary policy cannot give: we compute an exact global diagonalization of a bilinear network, remove 62% of its hidden dimensions with under 1% accuracy loss, and show that a global importance ranking beats a per-layer one. We also give first evidence that this global low-rank structure survives through attention and residual connections.

1 Introduction

Interpreting a large policy is hard largely because the network is not compositional. Nonlinear operations such as the softmax in attention, the smooth activation in an MLP, and layer normalization prevent us from rewriting the model as a single algebraic formula, so we are forced to study its behaviour on data rather than its weights directly.

A recent line of work argues that this is a design choice, not a necessity. If every learned operation is kept polynomial in the activations, meaning it is built only from linear maps, element-wise multiplications, additions, fixed masks and permutations, learned constants, and fixed scalar or diagonal rescalings, then the model is a structured polynomial. Once token identities are written as one-hot vectors, that polynomial is a single multilinear object, commonly called a tensor network, and tensor networks can be decomposed, ranked, and truncated using tools from numerical linear algebra applied to the weights alone.

Our contribution is an integration insight rather than a new primitive: the parts needed to build a fully tensor-decomposable Vision-Language-Action model (a VLA, a model that maps camera images and a language instruction to robot actions) already exist, and OpenVLA’s architecture is unusually ready to receive them. OpenVLA [1] already separates a VLA into four learned stages: two complementary image encoders (DINOv2 and SigLIP), a small projector that fuses them, a language backbone (Llama 2), and an action decoder. Each stage has a published polynomial replacement:

- **Bilinear feed-forward layer** (Riggs [2]). Replace the gated MLP $D(\text{swish}(Lx) \otimes Rx)$ with the activation-free $D[(L\bar{x}) \otimes (R\bar{x})]$, where $\otimes$ is the element-wise (Hadamard) product and $\bar{x}$ is defined below. This is exactly a low-rank factorization (a CP, or canonical polyadic, factorization) of a third-order weight tensor.
- **Softmax-free bilinear attention** (Riggs [2]). Replace the softmax attention pattern with the element-wise product of two separate learned query-key score matrices.
- **Foldable normalization** (Dooms et al. [3]). Normalize during training by dividing by a running average magnitude (a single scalar), then after training freeze that scalar and multiply it into the next weight matrix, so the deployed model contains no division at all.
- **Linear action head**. Replace the discretized action tokens and language-model-style decoding with a direct linear map from learned action-query states to a continuous action vector.

We assemble these along the OpenVLA skeleton and keep only genuinely non-learned interfaces outside the tensor-network boundary: string tokenization, camera decoding, safety clipping, and motor communication. The result, χ -VLA, is a policy whose learned computation from pixels and token identities to a continuous action is one tensor network. This paper reports the first implementation, the experiments that validate its parts, and, in depth, the property that motivates the whole exercise: the exported model can be exactly decomposed and compressed.

2 Background: the three polynomial primitives

The homogeneous coordinate. Throughout, we write $\bar{x} = [1; x]$ for the input with a constant 1 prepended. This “homogeneous coordinate” lets a bilinear layer represent not only quadratic terms but also linear terms and constants (a bias becomes the weight on the constant coordinate). Without it, a stack of L multiplicative layers would be forced to produce only degree- 2^L terms; with it, the network can represent every degree up to 2^L , including the near-linear functions that most of a real task needs.

Ordinary component	χ -VLA replacement	Algebraic form
Smooth activation (ReLU, GELU)	element-wise product of two linear maps	$(Lx) \otimes (Rx)$
Gated MLP	bilinear (CP) feed-forward	$D[(L\bar{x}) \otimes (R\bar{x})]$
Softmax attention	product of two score matrices	$C[M \otimes (A_1 \otimes A_2)/s]V$
Layer/RMS normalization	foldable scalar normalization	x/c with c a frozen scalar
MLP projector	cross-bilinear (CP) projector	$W_s s + W_m m + D_p[(L_s \bar{s}) \otimes (R_m \bar{m})]$
DINOv2 / SigLIP encoders	two polynomial χ -ViTs	(all blocks bilinear)
Llama 2 backbone	polynomial transformer	(all blocks bilinear)
Discrete action tokens	linear continuous-action head	$\hat{a}_k = W_a h_k + b_a$
Bias terms	constant (homogeneous) coordinate	$\bar{x} = [1; x]$

Table 1: Component-level replacements. A bar denotes the homogeneous coordinate $\bar{x} = [1; x]$; $\otimes$ is the element-wise product.

Bilinear feed-forward layer (a CP-factorized cubic tensor). With $L, R \in \mathbb{R}^{r \times (d+1)}$ and $D \in \mathbb{R}^{d \times r}$,

$$\text{BFFN}(x) = D[(L\bar{x}) \otimes (R\bar{x})], \quad y_o = \sum_{i,j} \left(\underbrace{\sum_r D_{or} L_{ri} R_{rj}}_{T_{oij}} \right) \bar{x}_i \bar{x}_j. \quad (1)$$

The middle expression makes the tensor structure explicit: the layer is exactly a third-order weight tensor $T \in \mathbb{R}^{d \times (d+1) \times (d+1)}$ contracted twice with the input. The factored form uses about $3dr$ parameters instead of the d^3 of the dense tensor, which is why it is called a CP factorization (a sum of r rank-one terms).

Softmax-free bilinear attention. Each head forms two query-key systems $A_1 = Q_1 K_1^\top$ and $A_2 = Q_2 K_2^\top$ and one value matrix V . The attention pattern is their element-wise product, with no softmax:

$$Y = C[M \otimes (A_1 \otimes A_2)/s]V. \quad (2)$$

Here M is a fixed mask (for example, lower-triangular for causal models), C is a fixed per-row rescaling $C_{ii} = 1/\sqrt{n_i}$ with n_i the number of visible keys, and s is a fixed scale. Because there is no softmax, the whole operation is polynomial in the activations. It can be evaluated three mathematically identical ways: the explicit $N \times N$ pattern; a ‘‘Khatra-Rao’’ form that avoids the $N \times N$ matrix by building row-wise Kronecker features; and a causal prefix-scan. We verified all three agree to 10^{-5} in single precision.

Foldable normalization. During training we divide activations by a single running scalar equal to their average root-mean-square magnitude. After training we re-estimate that scalar on a calibration set, freeze it, and multiply it into the following weight matrix ($W' = W/c$ before a linear map, or $L' = L/c$, $R' = R/c$ before a bilinear layer). A frozen scalar division is a fixed rescaling, so the deployed model has no operation whose divisor depends on the input. This is what we mean by ‘‘fold’’: the normalization module disappears into the weights.

3 From OpenVLA to χ -VLA

Figure 1 contrasts the two pipelines and Table 1 lists the block-level replacements. The high-level flow is deliberately preserved, while every learned internal block is replaced by a polynomial one.

4 Making the foldable model stable

The subtle question is not whether the model can be written as a tensor network (by construction it can) but whether the foldable normalization is stable inside a deep stack. A per-token normalization, which divides each token by its own magnitude, keeps every layer’s activations at unit scale on every input and is therefore very stable, but its divisor depends on the input, so it cannot be folded. A scalar normalization divides every input by the same frozen constant c , which is foldable, but a fixed constant cannot rescale an input that happens to be larger or smaller than average. Because the bilinear feed-forward layer is degree two, meaning it multiplies activations together, an input that arrives at twice the average scale produces an update about four times too large, and this error grows with depth.

The resolution is to keep the residual connections close to the identity. We give each residual branch a small learned gain (initialized at 0.01), so each layer changes the running representation only slightly and the degree-two amplification never compounds. We train a stable per-token model first and then distill it into a scalar-normalization student with these small gains. On a language model this student matches its teacher (validation loss 5.04 versus 5.0) and folds to a normalization-free graph with error about 10^{-6} in single precision and 10^{-12} in double precision.

We also measured why this works: across roughly 1.6 million tokens, the largest per-token magnitude at any normalization site is only about 2.8 times the average (the tail is thin), and it does not grow with depth. With the small gains, the per-layer amplification factor stays below one, so the fixed scalar is an accurate stand-in for the per-token normalization.

5 Exact global decomposition (ODT), in depth

This section is the payoff of the whole design and we treat it carefully. An ordinary softmax-and-normalization policy cannot be turned into a single algebraic object, so it cannot be globally diagonalized; a tensor network can.

5.1 What ODT is, and why it needs a tensor network

Suppose we want to compress a network by discarding directions in one of its hidden spaces, keeping only the ones that matter for the output. Call the hidden space we are analyzing a “bond”. The naive approach is a local one: take the weight matrix that reads from that bond, run a singular value decomposition (SVD), and keep its top directions. This is greedy and per-layer; it only sees the one adjacent weight matrix, not what the rest of the network does with those directions downstream.

Orthogonalize-Diagonalize-Truncate (ODT), introduced for tree-shaped bilinear networks by Dooks et al. [3], instead ranks bond directions by their *global* importance, accounting for the entire rest of the network, which we call the “environment” of the bond. This is only possible if the whole model is one multilinear object, because only then can the environment be contracted into a single operator whose singular directions are the true global importances. The three steps are:

1. **Orthogonalize.** Sweep through the network and replace each weight core by an isometry (an orthonormal map) times a small triangular remainder, using a reduced RQ decomposition of the matricized core, pushing the remainder toward the output. After this sweep each bond has an orthonormal basis, so importance can be read off directly as singular values rather than being tangled up with the basis.

2. **Diagonalize.** At a bond, form the Gram matrix of the environment, $G = Q^*Q$, where Q is the downstream map from the bond to the output. Its eigenvalues are the squared global importances of the bond directions and its eigenvectors are those directions.
3. **Truncate.** Keep the top k eigenvectors, insert the projector $P_k = V_k V_k^\top$ at the bond, and absorb it into the adjacent cores. The output error is bounded by the square root of the sum of the discarded eigenvalues, so the spectrum tells you in advance how much you can remove.

5.2 How we compute it for a bilinear chain

Our exact-decomposition model is a feed-forward bilinear network we call a χ -MLP, chosen because a chain with no attention and no residual connection is a clean tree, exactly the setting ODT was designed for. After folding, it is

$$z_0 = [1; Ex], \quad a_i = C_i(z_{i-1}, z_{i-1}), \quad z_i = [1; a_i], \quad \text{logits} = W_{\text{head}} a_L + b, \quad (3)$$

where E is a linear input embedding, each $C_i \in \mathbb{R}^{d \times (d+1) \times (d+1)}$ is a symmetric bilinear core recovered from the folded layer, and $C_i(z, z)$ means contracting the core twice with the homogeneous activation z .

To rank the directions of a bond a_i by global importance we build the exact downstream map from a_i to the output and contract it against itself over the output index and over every environment leg, leaving the bond leg open on both sides. For the deepest bond (the output of the first layer, whose downstream is two more bilinear layers) the downstream is a degree-four form in z_i . We construct it explicitly by composing the cores,

$$D[c, i, j, k, l] = \sum_{p, q} C_L^{\text{head}}[c, p, q] Z_2[p, i, j] Z_2[q, k, l], \quad G[i, i'] = \sum_{c, j, k, l} D[c, i, j, k, l] D[c, i', j, k, l], \quad (4)$$

where Z_2 expresses the second-layer homogeneous activation as a quadratic form in z_i and C_L^{head} folds the linear head into the last core. The $d \times d$ block of G over the non-constant coordinates gives the global importances. The local baseline uses only the adjacent core (the layer that directly reads the bond), ignoring everything downstream. Truncation projects the activation onto the top k eigenvectors and we measure the resulting accuracy.

5.3 Results on the χ -MLP

We train a three-layer χ -MLP (bond width 32) on the SVHN digit dataset to 0.81 accuracy and run the exact procedure.

- **Exact reconstruction.** The unrolled tensor network reproduces the trained module to 5×10^{-12} in double precision, confirming that the exported cores are the model, not an approximation of it.
- **Global low-rank structure.** Ranking the deepest bond by the global Gram, 20 of its 32 hidden dimensions (62%) can be removed with at most a 1% accuracy drop. Dooms et al. report about 70% on their own SVHN network; we report our measured figure rather than assuming their number transfers.
- **Global beats local.** At a matched number of kept directions, the global ranking preserves accuracy better than the local per-layer SVD, most clearly at the compression elbow.

retained directions k	4	6	8	12	16	
global ODT	0.571	0.777	0.797	0.809	0.811	(full width 32 gives 0.808; deepest bond)
local SVD	0.523	0.686	0.798	0.807	0.811	

The reason global wins is exactly the reason it needs a tensor network: at $k = 6$ the global ranking already knows which directions the two downstream layers will actually use, while the local ranking only sees the immediately following layer and keeps directions that later turn out not to matter.

5.4 From compression to interpretation: are the ranked modes real?

Removable-dimension counts show that low-rank structure *exists*; they do not show that the surviving directions are human-interpretable, nor that the ranking is *causal* rather than merely a good compressor. Déhérand [4] takes the complementary step, extending the χ -net idea to convolutional networks and reading each class output as a quadratic form whose eigenvectors, projected to pixels, are spatially coherent digit strokes. Two things are left open there: the interpretation is run only on a shallow single-bilinear layer, and visual coherence is never tied to an intervention. We close both on our own models.

Output-conditioned atoms. For a single-bilinear χ -classifier each class logit is *exactly* a quadratic form in the homogeneous embedded input, $\ell_c(x) = z^\top Q_c z + b_c$ with $z = [1; Ex]$ and $Q_c = \sum_o W_{c,o}^{\text{head}} C_o$ the head folded into the core (reconstruction against the trained module is 2.6×10^{-13} on SVHN, 0.80 accuracy). Eigendecomposing $Q_c = \sum_i \lambda_i v_i v_i^\top$ writes the logit as $\sum_i \lambda_i (v_i \cdot z)^2 + b_c$: signed “atoms” where $\lambda_i > 0$ supports class c and $\lambda_i < 0$ suppresses it, each projected to a pixel sensitivity map through E .

Coherence, honestly. The atoms concentrate energy centrally, where SVHN digits sit, in oriented stroke-like patterns; but their spatial locality (fraction of energy in the top 10% of pixels) is 0.63 versus 0.55 for random directions — only 1.16 $\times$. Our dense-bilinear atoms are therefore far less localized than Déhérand’s convolutional ones. This is direct support for his thesis that it is the *topology* (local connectivity, weight sharing), not tensor-convertibility alone, that makes the spectral modes crisply human-readable.

Faithfulness: the ranking is causal. We intervene on the extracted modes and measure accuracy: *keep* only the top- r modes of every class, *drop* the top- r (project them out), or drop r *random* modes.

modes r (of 65)	4	8	16	32	
keep top- r	0.398	0.663	0.779	0.804	(SVHN, full model 0.80)
drop top- r	0.639	0.442	0.275	0.212	
drop random- r (control)	0.785	0.690	0.643	0.591	

Removing the 16 top-ranked modes collapses accuracy to 0.28, while removing 16 *random* modes leaves 0.64: the spectral ranking is load-bearing, not an arbitrary basis. Amplifying a single top positive atom of a target class ($3\times$ its eigenvalue) raises that class logit by $+2.5$ to $+4$ and flips up to 12% of samples to the target — a directed, predicted-and-observed causal effect, the intervention Déhérand’s visual eigenspectra never demonstrated. The same test on the deep three-layer χ -MLP, ranking the deepest bond by the global Gram versus a random subspace of matched size, gives

0.78 vs 0.46 at $k = 6$ and beats random at every retained width below full rank — extending the causal-ranking result from a shallow model to a deep one.

5.5 Does this survive attention and residual connections?

The χ -MLP is feed-forward; a transformer adds attention (which mixes information across token positions) and residual connections. Extending exact, weight-only global ODT to a general attention network is an open problem, because the attention pattern is itself a learned degree-four function of the inputs, so the environment is no longer a fixed operator. We give the first empirical evidence that the structure is nonetheless there.

We train a strict, foldable transformer language model (using the teacher-then-distill recipe so it is stable) and analyze the bond at the block input, whose downstream *crosses* the attention layer (each token feeds the queries, keys, and values of every position), then the residual path, the feed-forward layer, and the head. Because the attention pattern is data-dependent here, we rank directions by a data-driven version of the global Gram, the average outer product of the output gradient at the bond, which measures how much each direction of the bond actually affects the loss through the full downstream including attention. The local baseline is the SVD of the adjacent input projections (the query, key, value, and feed-forward input weights).

	retained k	global (output-aware)	local (weight SVD)
1 attention layer (full-rank loss 5.23)	32	6.11	7.88
	64	5.47	6.27
	96	5.27	5.55
2 attention layers (full-rank loss 5.98)	32	6.18	7.78
	64	6.02	6.52
	96	5.99	6.18

The global ranking beats the local one at every retained width for both models, and the attention model still folds to an exact tensor network (double-precision fold error at machine precision). Notably the two-layer model is *more* compressible (64 of 128 directions, 50%, removable at under 0.05 nats) than the one-layer model (25%), which suggests deeper attention stacks have more, not less, global low-rank structure. Two honest caveats. First, this uses the data-driven global Gram; the fully weight-based, data-free contraction across an attention-crossing bond, and the transfer of the tree error bound, remain open. For bonds whose downstream is feed-forward (for example, the bond after the attention sub-layer), the exact weight-based procedure of the previous subsection applies directly. Second, the strict scalar-normalization model needs the distillation recipe to train stably beyond one layer.

5.6 Cost and limits

For the tree network the procedure costs about $O(Lh^4)$ time with $O(h^3)$ memory in the bond width h , which is why our exact-decomposition models use a small width. Unrolling a full attention transformer into one tensor over all token positions grows combinatorially, so exact whole-model contraction is only feasible at small scale; the practical path is to decompose one analysis bond at a time, as above.

6 Full-stack results

We built the model from the bottom up under a hard compute limit (single consumer-grade GPUs, no data-center GPUs), so the numbers below are on small models and modest data. The aim is a working, verified demonstration of every claim, not a scale record. All the polynomial primitives are shared across the language, vision, and action models.

Softmax-free vision encoder. A bilinear vision transformer (χ -ViT, 6.7 million parameters) on SVHN versus a parameter-matched baseline that differs only by using ordinary softmax attention:

attention	test accuracy	fraction of baseline
softmax (baseline)	0.928	(reference)
bilinear (χ-ViT, tensor-pure)	0.917	98.8%

Removing the softmax costs 1.2 accuracy points on real images.

Cross-bilinear projector. The projector fuses the two vision streams so that products between them are represented explicitly. On a task whose label is, by construction, the result of multiplying the two streams (and therefore cannot be separated by any purely additive model), at matched parameters:

fusion method	test accuracy	
cross-bilinear projector (tensor-pure)	0.971	
same, with the product term set to zero	0.161	(chance is 0.10)
concatenate then linear	0.160	
concatenate then MLP with GELU (not tensor-pure, matched size)	0.590	

The product term is worth 81 accuracy points over the concatenation baseline, and removing it erases exactly that gain. The tensor-pure projector even beats a non-polynomial MLP of the same size here.

End-to-end policy and language grounding. A full χ -VLA (a χ -ViT front-end, a joint bilinear transformer, and a linear action head) trained by behaviour cloning. On a controlled synthetic reaching task where the instruction names which of several objects to reach, the policy is 22 times more accurate than a language-blind baseline that always reaches the average object location, and shuffling the instruction makes its error 13 times worse, which shows it reads the instruction and grounds it in the image.

On real robot-imitation data (LIBERO-Object, ten tasks that each name a different object, offline behaviour cloning over 18,900 action chunks), we report the result carefully because the absolute error is easy to misread.

model input	normalized action error
full model (image + instruction + state)	0.021
correct image and state, shuffled instruction	0.194
predict the mean action (trivial baseline)	1.025
image and instruction zeroed (out of distribution)	1.148
image zeroed, instruction kept (out of distribution)	1.100

This table is easy to misread, so we are explicit. Actions are normalized to unit variance, so the mean-action baseline scores about 1.0 automatically and a comparison against it should not be read as strong grounding. We first checked whether the low error was the model coasting on the proprioceptive state, since robot trajectories are smooth. It is not: zeroing the image (with or without the instruction) sends the error above 1.0, so the model genuinely relies on its visual input rather than predicting the action from state alone. Note these zeroed-input rows are out-of-distribution inputs for a model trained on real images, so they are stress tests, not fair “state-only” baselines; a fair state-only baseline would require training a separate state-only model, which we did not do. The clean, in-distribution measure of language use is the shuffled instruction: keeping the correct image and state but supplying a mismatched instruction raises the error from 0.021 to 0.194, about nine times worse. In other words the policy identifies the correct target object from the image and the instruction together, and once the target is known the smooth action toward it is easy to predict, which is why the absolute error is small. We report offline action error only; closed-loop success on a simulator is deferred, and a low error does not by itself imply task success.

Reference configuration and attention kernel. The parameter count of the specification’s 450-million-parameter reference configuration (two twelve-layer vision encoders, a twenty-layer joint backbone, a 32,000-word vocabulary) works out to 448.2 million, matching the intended breakdown. Training it needs data-center GPUs and large-scale data and is deferred. We also measured that the explicit attention kernel is faster than the Khatri-Rao form for all sequence lengths up to 4096 on our hardware, so the policy’s short sequences (roughly one hundred to three hundred tokens) should use the explicit kernel; the factored form’s theoretical advantage needs a fused kernel or much longer sequences.

7 The experiment program: one substitution per experiment

The replacement table (Table 1) suggests its own experiment program, because each substitution can be tested on its own. Experiments 1 to 6 each change exactly one row of the table and hold everything else fixed, so any change in performance is attributable to that single substitution. Experiment 7 tests the reason for doing all the substitutions at once, namely that the resulting model can be globally decomposed. Table 2 lists the program and its status; the results above cover experiments 1, 3, 5, and 7.

8 Discussion and limitations

By construction the χ -VLA forward map is a tensor network, and we verify this end to end for the language backbone, a one-layer and a two-layer attention model, and the feed-forward decomposition model. The central new finding is that global low-rank structure, the property that makes ODT worthwhile, is present not only in feed-forward chains but also through attention and residual connections, and it grows rather than shrinks with depth in our small models.

The honest boundaries are as follows. The exact, weight-only, data-free version of global ODT is established here only for feed-forward downstream bonds; for attention-crossing bonds we currently use a data-driven importance measure, and the transfer of Doods et al.’s error bound to attention networks is not proven. The strict foldable model is stable and matches its non-strict teacher only in the small-residual-gain regime, which is a real constraint we did not need to pay any accuracy for at this scale but which may matter at larger scale. All experiments are small; the reference-scale

#	Substitution tested	Question	Status
1	softmax vs bilinear attention, crossed with gated vs bilinear MLP	Where is the accuracy cost, in the attention change or the MLP change?	done (language)
2	tied vs untied score systems	Do two independent query-key systems help, or is any degree-four pattern enough?	planned
3	cross-bilinear projector vs concatenate-then-linear	Does forcing the product help, or would the backbone recover it anyway?	done
4	homogeneous coordinate on or off	Does dropping the constant coordinate hurt generalization?	planned
5	foldable scalar vs per-input normalization	Is the foldable normalization enough to stay stable?	done
6	continuous vs discretized action head	Does the continuous regression cost anything?	planned
7	global ODT vs local SVD	Does the global low-rank structure exist, and does global beat local?	done (feed-forward and attention)

Table 2: The experiment program. Experiments 1 to 6 each isolate one row of Table 1; experiment 7 tests whether the combined payoff, global decomposability, actually appears.

model is specified and its size verified but not trained, for lack of data-center hardware. On the robot task we report offline action error and a language-grounding gap, not closed-loop success.

The interpretation results also point past the current architecture. Our atoms are causal but only mildly localized, and Déhérand’s convolutional comparison [4] shows why: the tensor-network *topology* decides whether the spectral modes are human-coherent, not the mere fact of tensor-convertibility. This suggests the next χ -VLA should build coherence into its structure rather than hope to read it out afterwards — for instance local or multiscale bilinear vision cores in place of dense ones, fixed structural tensors that separate world and robot geometry from learned policy, symmetry-aware canonicalization that groups translated copies of a mechanism into one orbit, and an output-conditioned, modality- and degree-resolved decomposition of continuous actions (which signed mechanism drives “move left”, and does it survive intervention). Déhérand also warns that a convertible network need not be tractably *contractible*; its unrolled tensor network can grow exponentially through fan-out, which argues for bounded-treewidth design (local attention, object-slot bottlenecks) from the start.

9 Conclusion

The parts needed for a fully tensor-decomposable VLA already exist, and OpenVLA’s modular design is ready to receive them. Assembling bilinear feed-forward layers, softmax-free bilinear attention, foldable normalization, and a linear action head gives χ -VLA, a policy that is a single tensor network from pixels and token identities to continuous actions. We showed it folds exactly, stays competitive after folding, benefits measurably from the explicit cross-stream product, uses language on a real robot dataset, and, uniquely among policy architectures, admits an exact global diagonalization in which most hidden dimensions can be removed and a global importance ranking beats a local one, including through attention.

References

- [1] A. Kim et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. arXiv:2406.09246, 2024.
- [2] L. Riggs. *Tensor-Transformer Variants are Surprisingly Performant*. LessWrong, 2024. Code: <https://github.com/loganriggs/modded-nanogpt>.
- [3] T. Dooms et al. *χ -nets: CP-factorized bilinear networks with foldable normalization and ODT analysis*.
- [4] A. Déhérand. *Convolutional Tensor Networks for Weight-Based Mechanistic Interpretability*. Master’s thesis, Vrije Universiteit Brussel, 2025–2026.



Figure 1: OpenVLA (left) versus χ -VLA (right), stage by stage. Green boxes are the polynomial (tensorized) replacements; blue boxes are the non-polynomial originals they replace; grey boxes are kept unchanged or lie outside the tensor-network boundary. Patch embedding and the tokenizer are already linear and need no change. The softmax, smooth-activation, and layer-normalization blocks (the dual vision encoders, the MLP projector, and the Llama 2 backbone) and the discrete action decoder are each swapped for a polynomial equivalent with foldable normalization. Camera decoding, string tokenization, and safety clipping stay external.